COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/16/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **IMPLEMENTING TREES**

1. PROCEDURE OUTPUT

Main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

int main(){

    //creating the root node:
    BinaryTree<char>* root = new BinaryTree<char>('A');

    //creating a child of node A:
    root -> insertLeftChild('B');
    root -> insertRightChild('C');

    //insert a child to node B;
    root->getLeftChild()->insertLeftChild('D');
    root->getLeftChild()->insertRightChild('F');

    //insert a child to node C;
    root->getRightChild()->insertLeftChild('E');

    //print rootnode
    std::cout << "Root: " << root->getRootVal() << std::endl;

    //children of root node
    std::cout << "LeftChild: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "RightChild: " << root->getRightChild()->getRootVal() << std::endl;

    //children of sub trees B and C
        std::cout << "Node B left Child: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
        std::cout << "Node B right Child: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->getLeftChild()->getRightChild()->setRootVal('E');
        std::cout << "Node B right Child: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

BinaryTree.h

```h
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key;
        //pointers
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        //constructor:
        BinaryTree(T rootObj);

        //inserting a child node
        void insertLeftChild(T newNode);
        void insertRightChild(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();

        T getLeftChildVal();

        void setRootVal(T obj);

        bool isleaf();
};

template <typename T>
BinaryTree<T>::BinaryTree(T rootObj){

    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;

}
```

```cpp
template <typename T>
void BinaryTree<T>::insertLeftChild(T newNode){
    leftChild = new BinaryTree<T>(newNode);

    if (leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    }else{
    //Create a new Node
    BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

    newChild ->leftChild = leftChild;
    leftChild = newChild;
    }
}
template <typename T>
void BinaryTree<T>::insertRightChild(T newNode){
    rightChild = new BinaryTree<T>(newNode);

    if (rightChild == nullptr){
        rightChild = new BinaryTree<T>(newNode);
    }else{
    //Create a new Node
    BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

    newChild ->rightChild = rightChild;
    rightChild = newChild;
    }
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild(){
    return rightChild;
}

template<typename T>
T BinaryTree<T>::getRootVal(){
    return key;
}

template<typename T>
void BinaryTree<T>::setRootVal(T obj){
    key = obj;
}
```

```
template<typename T>
bool BinaryTree<T>::isleaf(){
    return leftChild == nullptr && rightChild == nullptr;
}
#endif
```

Analysis:

The code snippets show how a binary tree is implemented. It has an insertion function and display function. However, instead of in linked list that has a head and tail, a binary tree has a left and right. The base parent is called the root, and the branches (left and right) are called the sub tree parents. Basically, child members can be implemented on these parent classes which can also have some offshoots if implemented. However, in this code snippet, only one of the two parent sub-trees has its own children. On the other hand, one doesn't have any which is why it is called a leaf member. The tree representation has a total of three levels, the top most part being the root, the 2nd layer being the 'B' and 'C' and the third one being the 'D' and 'E' that are offsprings of the node B.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY